

ODF Reconstruction from Pole Figure Data

R. Hielscher

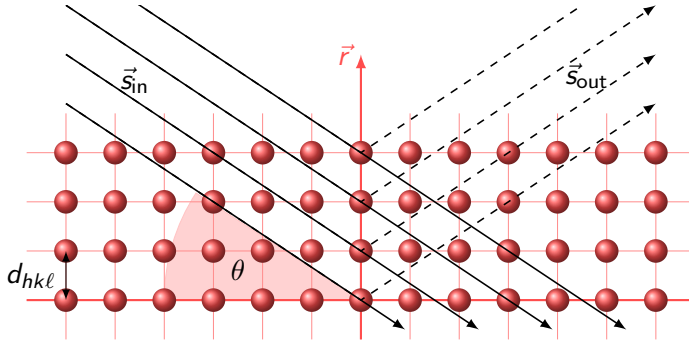
Faculty of Mathematics,
TU Bergakademie Freiberg, Germany

MTEX Workshop 2024

Braggs Law

- ▶ λ - wave length
- ▶ 2θ - angle between beam source $\vec{s}_{\text{in}}$ and detector $\vec{s}_{\text{out}}$
- ▶ $d_{hkl} = 1/\|\{hkl\}\|_2$ - distance between the lattice planes $\{hkl\}$

$$2d_{hkl} \sin \theta = n\lambda, \quad n = 1, 2, \dots$$

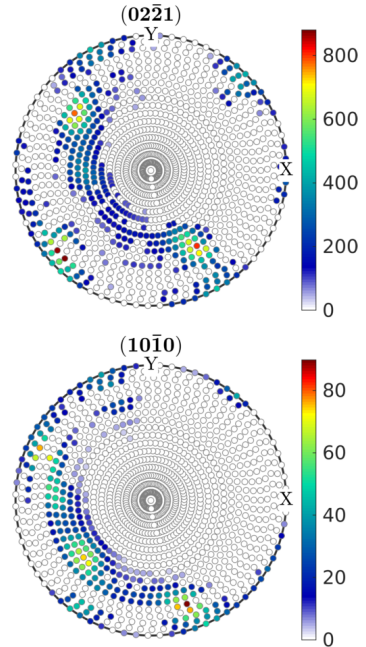


Diffraction Pole Figures

- ▶ For a fixed wavelength λ and angle 2θ there is often only a single crystal direction $\vec{h} = (hkl)$ such that Braggs law is satisfied for the corresponding d_{hkl} .
- ▶ The intensity measured at the detector is proportional to the irradiated volume of crystals that are orientated such that the lattice planes (hkl) are perpendicular to $\vec{r} = \vec{s}_{in} - \vec{s}_{out}$
- ▶ This corresponds to all grains with orientation $\mathbf{O}$ such that

$$\mathbf{O}\vec{h} = \vec{r},$$

i.e. all orientations that appear in the $\vec{h} = (hkl)$ -pole figure at position $\vec{r} = \vec{s}_{in} - \vec{s}_{out}$.

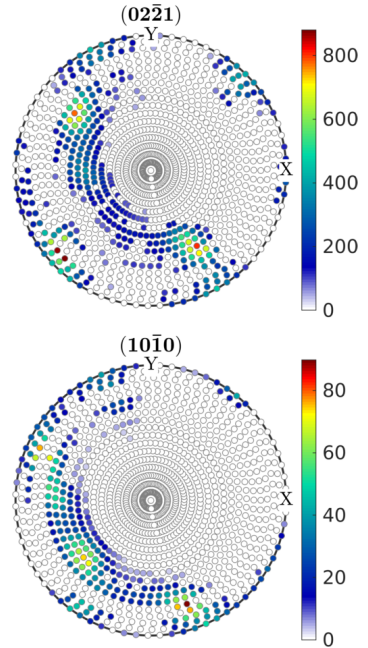


Diffraction Pole Figures

- ▶ For a fixed wavelength λ and angle 2θ there is often only a single crystal direction $\vec{h} = (hkl)$ such that Bragg's law is satisfied for the corresponding d_{hkl} .
- ▶ The intensity measured at the detector is proportional to the irradiated volume of crystals that are orientated such that the lattice planes (hkl) are perpendicular to $\vec{r} = \vec{s}_{\text{in}} - \vec{s}_{\text{out}}$
- ▶ This corresponds to all grains with orientation $\mathbf{O}$ such that

$$\mathbf{O}\vec{h} = \vec{r},$$

i.e. all orientations that appear in the $\vec{h} = (hkl)$ -pole figure at position $\vec{r} = \vec{s}_{\text{in}} - \vec{s}_{\text{out}}$.

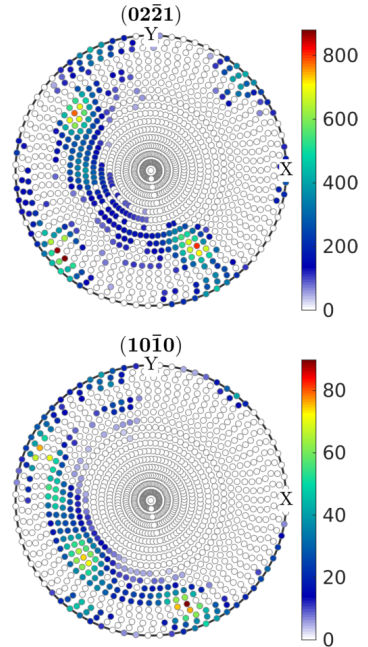


Diffraction Pole Figures

- ▶ For a fixed wavelength λ and angle 2θ there is often only a single crystal direction $\vec{h} = (hkl)$ such that Bragg's law is satisfied for the corresponding d_{hkl} .
- ▶ The intensity measured at the detector is proportional to the irradiated volume of crystals that are orientated such that the lattice planes (hkl) are perpendicular to $\vec{r} = \vec{s}_{\text{in}} - \vec{s}_{\text{out}}$
- ▶ This corresponds to all grains with orientation $\mathbf{O}$ such that

$$\mathbf{O}\vec{h} = \vec{r},$$

i.e. all orientations that appear in the $\vec{h} = (hkl)$ -pole figure at position $\vec{r} = \vec{s}_{\text{in}} - \vec{s}_{\text{out}}$.



The Radon Transform

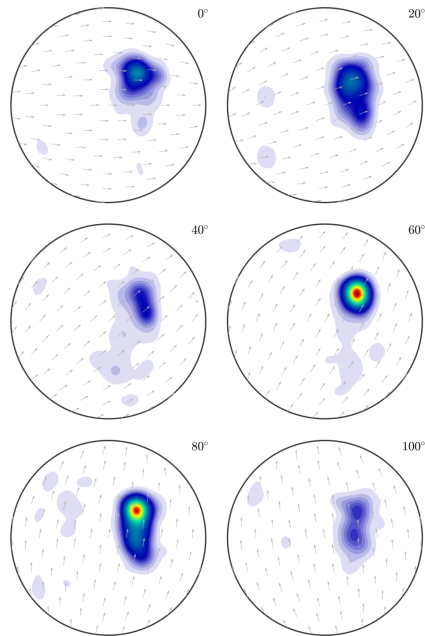
- ▶ detector position $\vec{r} = \vec{s}_{\text{in}} - \vec{s}_{\text{out}}$
- ▶ active lattice plane $\vec{h} = (hkl)$
- ▶ For N grains with orientations $\mathbf{O}_n$ and volumes v_n , $n = 1, \dots, N$ the intensity is proportional to

$$I(\vec{h}, \vec{r}) \sim \sum_{n: \mathbf{O}_n \vec{h} = \vec{r}} v_n$$

- ▶ For orientations distributed according to the ODF f the intensity is proportional to

$$I(\vec{h}, \vec{r}) \sim \mathcal{R}f(\vec{h}, \vec{r}) = \int_{\mathbf{O} \vec{h} = \vec{r}} f(\mathbf{O}) d\mathbf{O}$$

- ▶ The integral is called Radon transform of the ODF f and $\mathcal{R}f(\vec{h}, \cdot)$ the (hkl) pole density function.



The Radon Transform

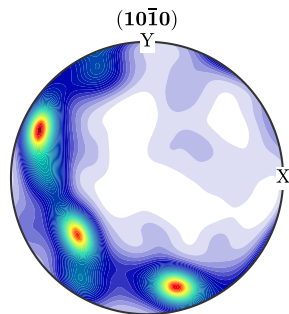
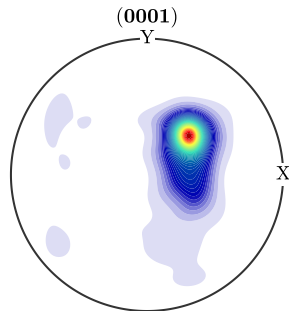
- ▶ detector position $\vec{r} = \vec{s}_{\text{in}} - \vec{s}_{\text{out}}$
- ▶ active lattice plane $\vec{h} = (hkl)$
- ▶ For N grains with orientations $\mathbf{O}_n$ and volumes v_n , $n = 1, \dots, N$ the intensity is proportional to

$$I(\vec{h}, \vec{r}) \sim \sum_{n: \mathbf{O}_n \vec{h} = \vec{r}} v_n$$

- ▶ For orientations distributed according to the ODF f the intensity is proportional to

$$I(\vec{h}, \vec{r}) \sim \mathcal{R}f(\vec{h}, \vec{r}) = \int_{\mathbf{O} \vec{h} = \vec{r}} f(\mathbf{O}) d\mathbf{O}$$

- ▶ The integral is called Radon transform of the ODF f and $\mathcal{R}f(\vec{h}, \cdot)$ the (hkl) pole density function.



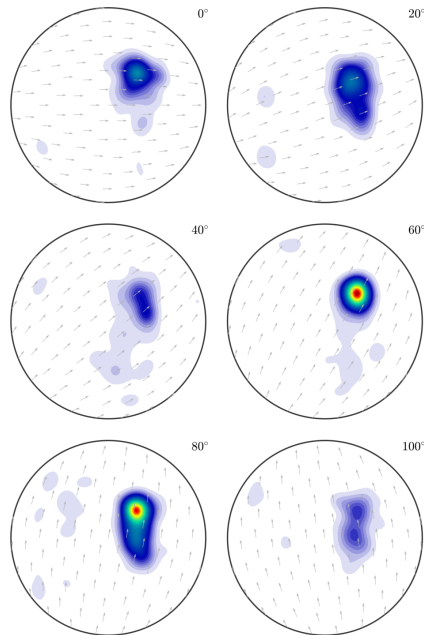
The Pole Density Function in MTEX

```
cs = crystalSymmetry.load('quartz')  
plot(odf, 'sigma')
```

```
h = Miller({0,0,0,1},{1,0,-1,0},cs)  
plotPDF(odf,h)
```

```
pdf = calcPDF(odf,h(2))  
[m,pos] = max(pdf,'numLocal',3)
```

```
pdf = calcPDF(odf,h(1))  
[m,pos] = max(pdf)  
f = fibre(h(1),pos)  
plot(odf,f)
```



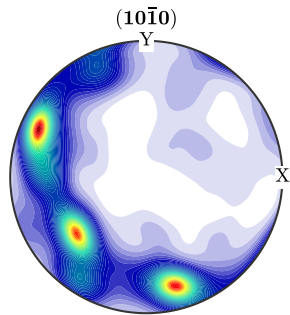
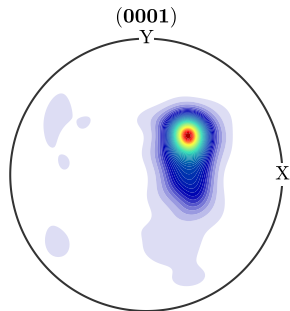
The Pole Density Function in MTEX

```
cs = crystalSymmetry.load('quartz')  
plot(odf, 'sigma')
```

```
h = Miller({0,0,0,1},{1,0,-1,0},cs)  
plotPDF(odf,h)
```

```
pdf = calcPDF(odf,h(2))  
[m,pos] = max(pdf,'numLocal',3)
```

```
pdf = calcPDF(odf,h(1))  
[m,pos] = max(pdf)  
f = fibre(h(1),pos)  
plot(odf,f)
```



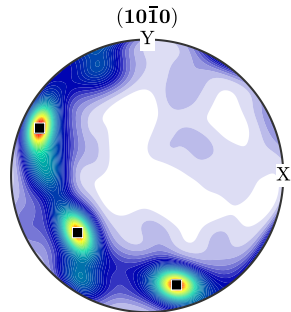
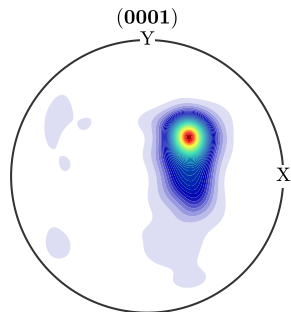
The Pole Density Function in MTEX

```
cs = crystalSymmetry.load('quartz')  
plot(odf, 'sigma')
```

```
h = Miller({0,0,0,1},{1,0,-1,0},cs)  
plotPDF(odf,h)
```

```
pdf = calcPDF(odf,h(2))  
[m,pos] = max(pdf,'numLocal',3)
```

```
pdf = calcPDF(odf,h(1))  
[m,pos] = max(pdf)  
f = fibre(h(1),pos)  
plot(odf,f)
```



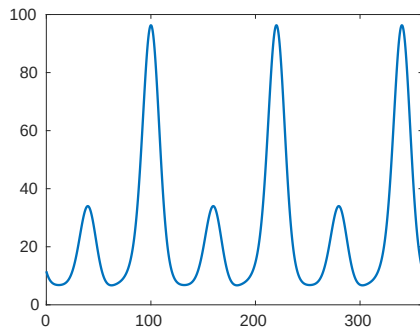
The Pole Density Function in MTEX

```
cs = crystalSymmetry.load('quartz')  
plot(odf, 'sigma')
```

```
h = Miller({0,0,0,1},{1,0,-1,0},cs)  
plotPDF(odf,h)
```

```
pdf = calcPDF(odf,h(2))  
[m,pos] = max(pdf,'numLocal',3)
```

```
pdf = calcPDF(odf,h(1))  
[m,pos] = max(pdf)  
f = fibre(h(1),pos)  
plot(odf,f)
```



Importing Pole Figure Data

```
CS = crystalSymmetry.load('quartz')
```

```
% lattice planes
```

```
h = {Miller(1,0,-1,0,CS), ...  
     Miller({0,1,-1,1},{1,0,-1,1},CS)}
```

```
% structure coefficients
```

```
c = {1,[0.52 ,1.23]};
```

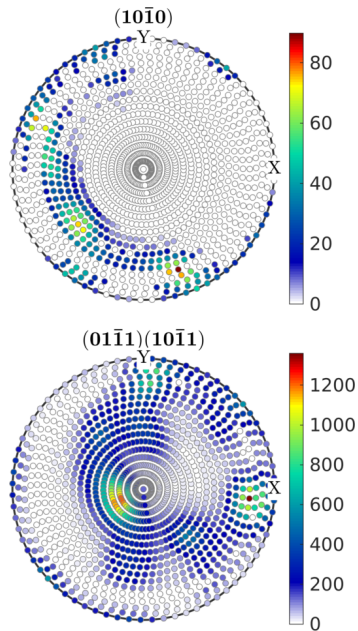
```
% import data
```

```
pf = PoleFigure.load(fnames,h,...  
                    'superposition',c)
```

```
pf = PoleFigure  
crystal symmetry : -3m1, X||a*, Y||b, Z||c*  
specimen symmetry: 1
```

```
h = (10-10), r = 72 x 19 points
```

```
h = (01-11)(10-11), r = 72 x 19 points
```



Modifying Pole Figure Data

rotate

```
pfRot = rot * pf
```

scale

```
pf = [1 0.1] * pf
```

normalize

```
pf = pf.normalize
```

properties

```
pf.intensities, pf.r, pf.h, pf.CS
```

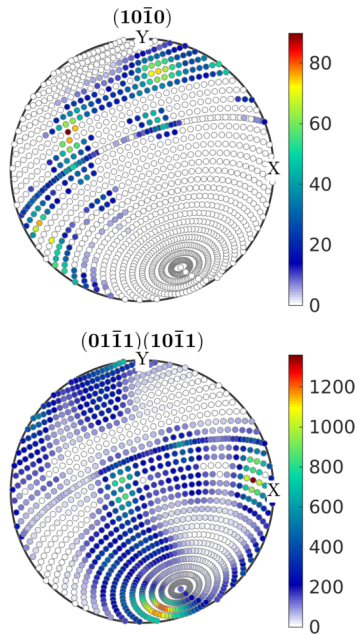
modify

```
pf(pf.intensities < 0) = 0
```

```
pf(pf.isOutlier) = []
```

first pole figure

```
pf{1}
```



Modifying Pole Figure Data

rotate

```
pfRot = rot * pf
```

scale

```
pf = [1 0.1] * pf
```

normalize

```
pf = pf.normalize
```

properties

```
pf.intensities, pf.r, pf.h, pf.CS
```

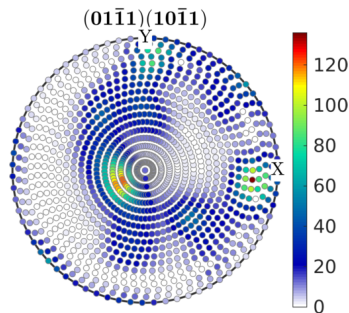
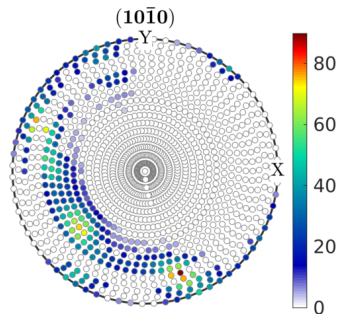
modify

```
pf(pf.intensities < 0) = 0
```

```
pf(pf.isOutlier) = []
```

first pole figure

```
pf{1}
```



Modifying Pole Figure Data

rotate

```
pfRot = rot * pf
```

scale

```
pf = [1 0.1] * pf
```

normalize

```
pf = pf.normalize
```

properties

```
pf.intensities, pf.r, pf.h, pf.CS
```

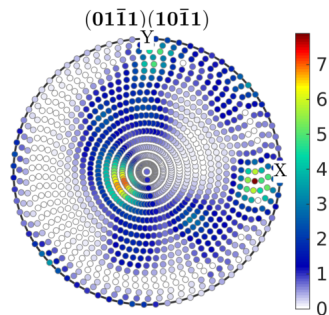
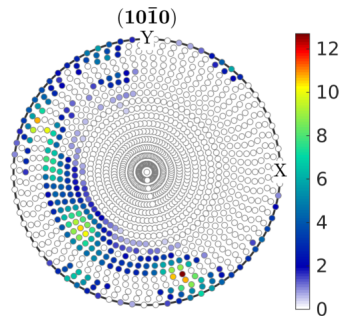
modify

```
pf(pf.intensities < 0) = 0
```

```
pf(pf.isOutlier) = []
```

first pole figure

```
pf{1}
```



Modifying Pole Figure Data

rotate

```
pfRot = rot * pf
```

scale

```
pf = [1 0.1] * pf
```

normalize

```
pf = pf.normalize
```

properties

```
pf.intensities , pf.r , pf.h , pf.CS
```

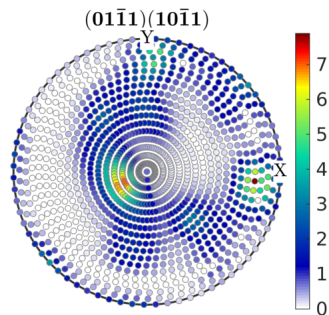
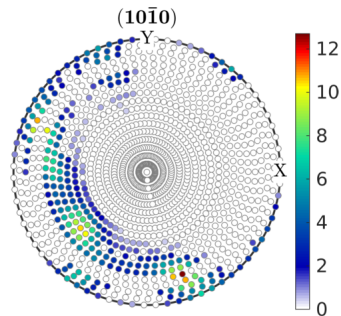
modify

```
pf(pf.intensities < 0) = 0
```

```
pf(pf.isOutlier) = []
```

first pole figure

```
pf{1}
```



Modifying Pole Figure Data

rotate

```
pfRot = rot * pf
```

scale

```
pf = [1 0.1] * pf
```

normalize

```
pf = pf.normalize
```

properties

```
pf.intensities , pf.r , pf.h , pf.CS
```

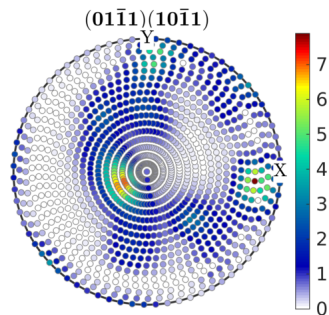
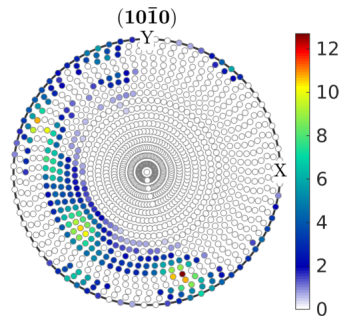
modify

```
pf(pf.intensities < 0) = 0
```

```
pf(pf.isOutlier) = []
```

first pole figure

```
pf{1}
```



Modifying Pole Figure Data

rotate

```
pfRot = rot * pf
```

scale

```
pf = [1 0.1] * pf
```

normalize

```
pf = pf.normalize
```

properties

```
pf.intensities , pf.r , pf.h , pf.CS
```

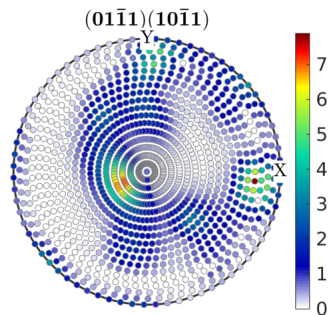
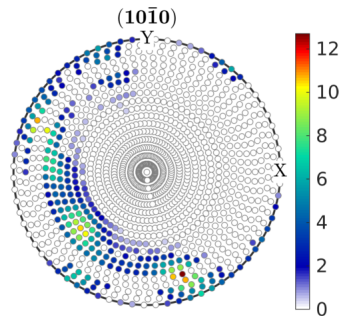
modify

```
pf(pf.intensities < 0) = 0
```

```
pf(pf.isOutlier) = []
```

first pole figure

```
pf{1}
```



Background and Defocusing Correction

the data

```
pf = PoleFigure
    crystal symmetry : m-3m
    specimen symmetry: 1

h = (104), r = 679 x 1 points
h = (104), r = 16 x 1 points
h = (110), r = 679 x 1 points
h = (110), r = 16 x 1 points
```

background correction

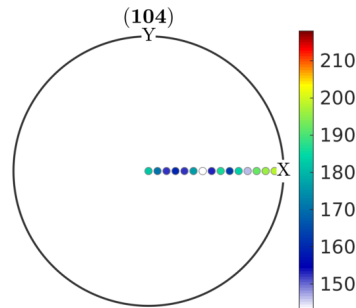
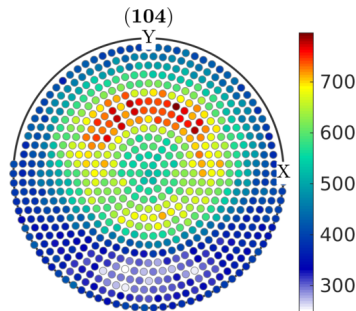
```
pf = correct(pf{[1,3]}, 'bg', pf{[2,4]})
```

defocusing correction

```
pf = correct(pf, 'bg', pf_bg, ...
            'def', def, 'defbg', pf_defbg)
```

by function

```
pf = pf .* myCorrectionFun(pf.r.theta)
```



Background and Defocusing Correction

the data

```
pf = PoleFigure
    crystal symmetry : m-3m
    specimen symmetry: 1

    h = (104), r = 679 x 1 points
    h = (104), r = 16 x 1 points
    h = (110), r = 679 x 1 points
    h = (110), r = 16 x 1 points
```

background correction

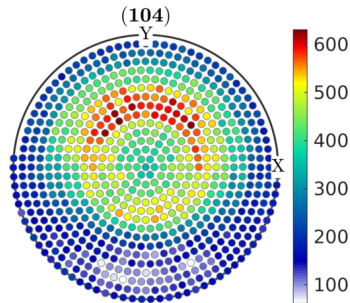
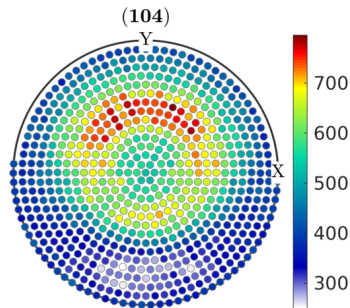
```
pf = correct(pf{[1,3]}, 'bg', pf{[2,4]})
```

defocusing correction

```
pf = correct(pf, 'bg', pf_bg, ...
    'def', def, 'defbg', pf_defbg)
```

by function

```
pf = pf .* myCorrectionFun(pf.r.theta)
```



Background and Defocusing Correction

the data

```
pf = PoleFigure
    crystal symmetry : m-3m
    specimen symmetry: 1

    h = (104), r = 679 x 1 points
    h = (104), r = 16 x 1 points
    h = (110), r = 679 x 1 points
    h = (110), r = 16 x 1 points
```

background correction

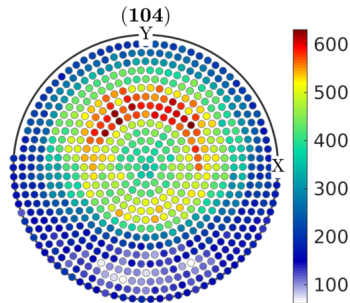
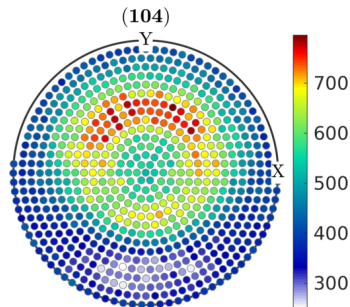
```
pf = correct(pf{[1,3]}, 'bg', pf{[2,4]})
```

defocusing correction

```
pf = correct(pf, 'bg', pf_bg, ...
    'def', def, 'defbg', pf_defbg)
```

by function

```
pf = pf .* myCorrectionFun(pf.r.theta)
```



Background and Defocusing Correction

the data

```
pf = PoleFigure
    crystal symmetry : m-3m
    specimen symmetry: 1

h = (104), r = 679 x 1 points
h = (104), r = 16 x 1 points
h = (110), r = 679 x 1 points
h = (110), r = 16 x 1 points
```

background correction

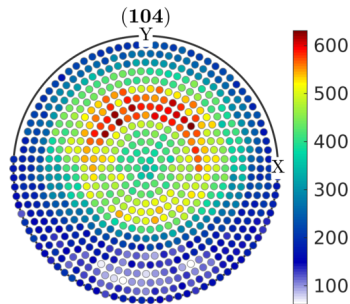
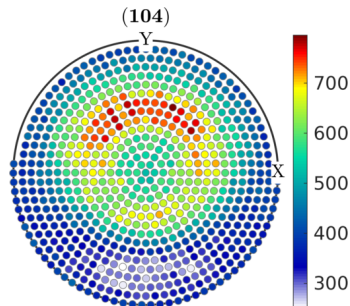
```
pf = correct(pf{[1,3]}, 'bg', pf{[2,4]})
```

defocusing correction

```
pf = correct(pf, 'bg', pf_bg, ...
    'def', def, 'defbg', pf_defbg)
```

by function

```
pf = pf .* myCorrectionFun(pf.r.theta)
```



ODF Reconstruction from Pole Figure Data

the data

```
pf = PoleFigure
```

```
crystal symmetry : Quartz (321, X||a*, Y||b, Z||c*)
```

```
specimen symmetry: 1
```

```
h = (02-21), r = 72 x 19 points
```

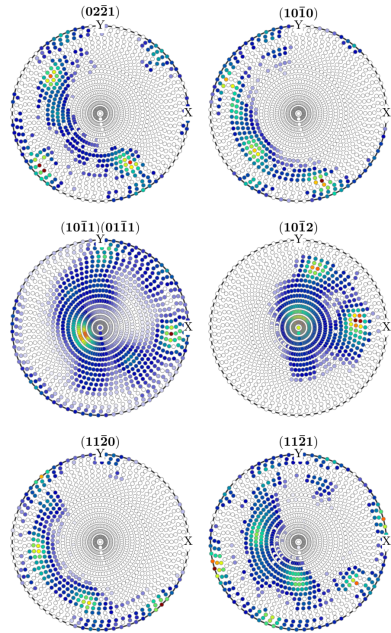
```
h = (10-10), r = 72 x 19 points
```

```
h = (10-11)(01-11), r = 72 x 19 points
```

```
h = (10-12), r = 72 x 19 points
```

```
h = (11-20), r = 72 x 19 points
```

```
h = (11-21), r = 72 x 19 points
```



ODF Reconstruction from Pole Figure Data

the data

```
pf = PoleFigure
    crystal symmetry : Quartz (321, X||a*, Y||b, Z||c*)
    specimen symmetry: 1

    h = (02-21), r = 72 x 19 points
    h = (10-10), r = 72 x 19 points
    h = (10-11)(01-11), r = 72 x 19 points
    h = (10-12), r = 72 x 19 points
    h = (11-20), r = 72 x 19 points
    h = (11-21), r = 72 x 19 points
```

0		0.89	0.91	0.77	1.12	0.90
1		0.87	0.87	0.66	1.02	0.90
2		0.84	0.82	0.54	0.93	0.88
3		0.76	0.73	0.43	0.83	0.83
4		0.67	0.66	0.41	0.76	0.79
5		0.65	0.63	0.40	0.74	0.78
6		0.64	0.62	0.39	0.73	0.77

options:

```
'halfwidth' 'kernel'
'iterMin' 'iterMax'
'resolution'
'zeroRange'
'noGhostCorrection'
```

odf reconstruction

```
odf = calcODF(pf)
```

```
odf = ODF
    crystal symmetry : Quartz (321, X||a*, Y||b, Z||c*)

    Radially symmetric portion:
        kernel: de la Vallee Poussin, halfwidth 5
        center: 19836 orientations, resolution: 5
        weight: 1
```

ODF Reconstruction from Pole Figure Data

the data

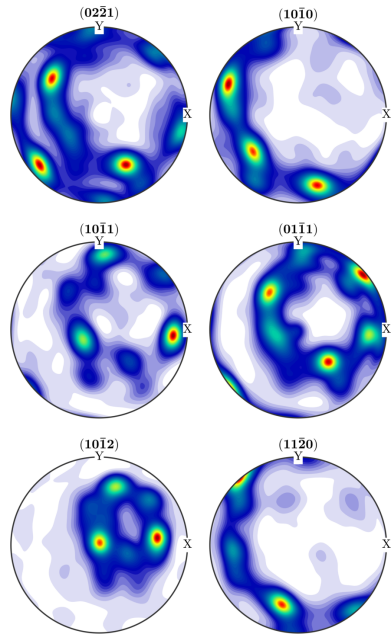
```
pf = PoleFigure
    crystal symmetry : Quartz (321, X||a*, Y||b, Z||c*)
    specimen symmetry: 1

    h = (02-21), r = 72 x 19 points
    h = (10-10), r = 72 x 19 points
    h = (10-11)(01-11), r = 72 x 19 points
    h = (10-12), r = 72 x 19 points
    h = (11-20), r = 72 x 19 points
    h = (11-21), r = 72 x 19 points
```

odf reconstruction

```
odf = calcODF(pf)
```

```
plotPDF(odf, pf.h(1:6))
```



The Ambiguity of the ODF Reconstruction Problem

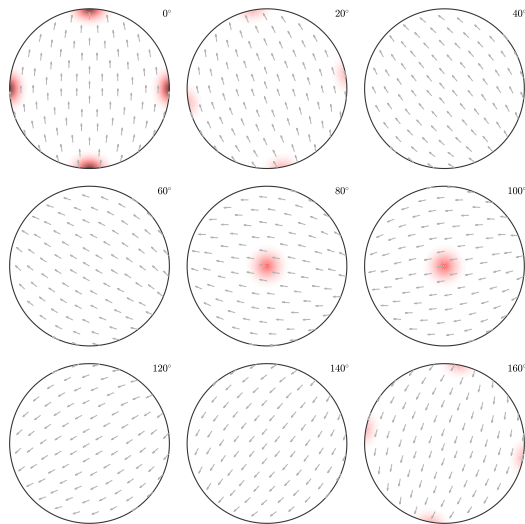
The following two ODFs

```
ax = [xvector, yvector, zvector]
ori2 = orientation.byAxisAngle(...
    ax, 90*degree, cs)
odf2 = unimodalODF(ori)
```

```
ax = vector3d(1,1,1)
ori2 = orientation.byAxisAngle(...
    ax, [0 120 240]*degree, cs)
odf2 = unimodalODF(ori)
```

have **7** identical pole figures

```
h = Miller({1,0,0},{0,1,0},...
    {0,0,1},{1,1,0},{1,0,1},...
    {0,1,1},{1,1,1}, cs)
plotPDF(odf1, h)
```



The Ambiguity of the ODF Reconstruction Problem

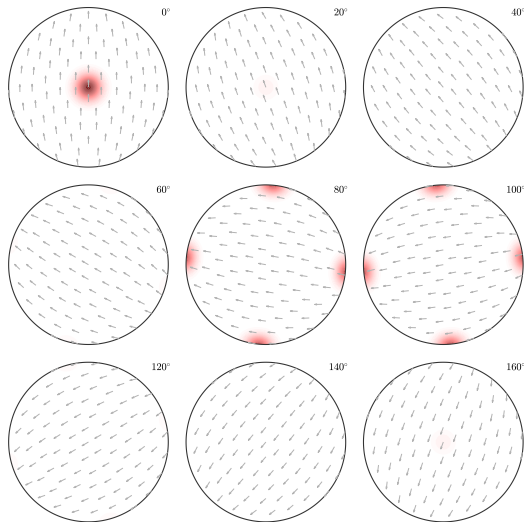
The following two ODFs

```
ax = [xvector, yvector, zvector]
ori2 = orientation.byAxisAngle(...
    ax, 90*degree, cs)
odf2 = unimodalODF(ori)
```

```
ax = vector3d(1,1,1)
ori2 = orientation.byAxisAngle(...
    ax, [0 120 240]*degree, cs)
odf2 = unimodalODF(ori)
```

have 7 identical pole figures

```
h = Miller({1,0,0},{0,1,0},...
    {0,0,1},{1,1,0},{1,0,1},...
    {0,1,1},{1,1,1}, cs)
plotPDF(odf1, h)
```



The Ambiguity of the ODF Reconstruction Problem

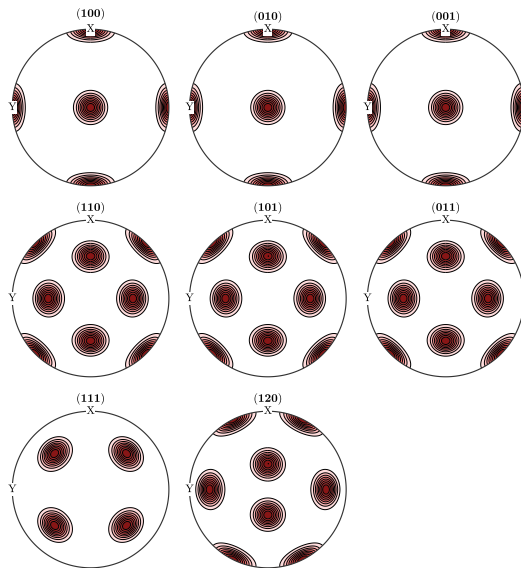
The following two ODFs

```
ax = [xvector, yvector, zvector]
ori2 = orientation.byAxisAngle(...
    ax, 90*degree, cs)
odf2 = unimodalODF(ori)
```

```
ax = vector3d(1,1,1)
ori2 = orientation.byAxisAngle(...
    ax, [0 120 240]*degree, cs)
odf2 = unimodalODF(ori)
```

have **7** identical pole figures

```
h = Miller({1,0,0},{0,1,0},...
    {0,0,1},{1,1,0},{1,0,1},...
    {0,1,1},{1,1,1}, cs)
plotPDF(odf1, h)
```



The Ambiguity of the ODF Reconstruction Problem

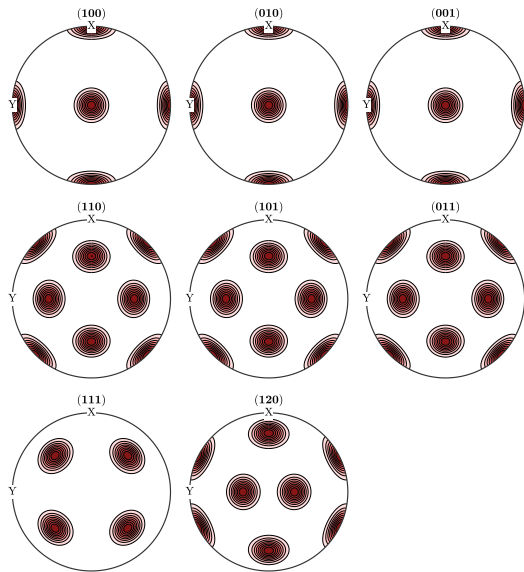
The following two ODFs

```
ax = [xvector, yvector, zvector]
ori2 = orientation.byAxisAngle(...
    ax, 90*degree, cs)
odf2 = unimodalODF(ori)
```

```
ax = vector3d(1,1,1)
ori2 = orientation.byAxisAngle(...
    ax, [0 120 240]*degree, cs)
odf2 = unimodalODF(ori)
```

have **7** identical pole figures

```
h = Miller({1,0,0},{0,1,0},...
    {0,0,1},{1,1,0},{1,0,1},...
    {0,1,1},{1,1,1}, cs)
plotPDF(odf1, h)
```



The Ambiguity of the ODF Reconstruction Problem

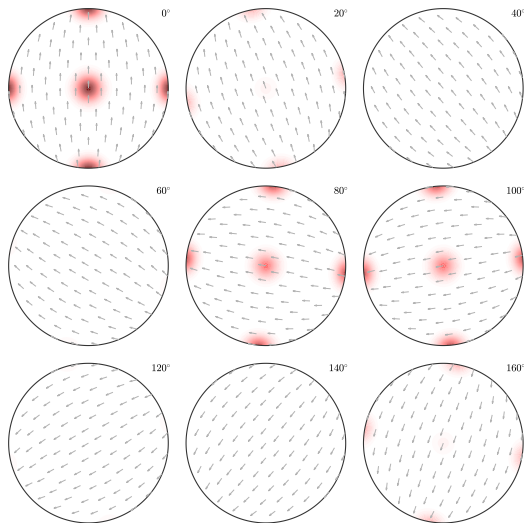
The following two ODFs

```
ax  = [xvector, yvector, zvector]
ori2 = orientation.byAxisAngle(...
    ax, 90*degree, cs)
odf2 = unimodalODF(ori)
```

```
ax  = vector3d(1,1,1)
ori2 = orientation.byAxisAngle(...
    ax, [0 120 240]*degree, cs)
odf2 = unimodalODF(ori)
```

have **7** identical pole figures

```
h = Miller({1,0,0},{0,1,0},...
    {0,0,1},{1,1,0},{1,0,1},...
    {0,1,1},{1,1,1},cs)
plotPDF(odf1, h)
```



The Even / Odd Problem

the famous SantaFe sample odf

```
plot(SantaFe, 'layout', [4 3])
```

simulate discrete pole figures

```
pf = calcPoleFigure(odf, h)
```

odf reconstruction without ghost correction

```
rec = calcODF(pf, 'noGhostCorrection')
```

odf reconstruction with ghost correction

```
rec6 = calcODF(pf)
```

odf reconstruction with 2 pole figures

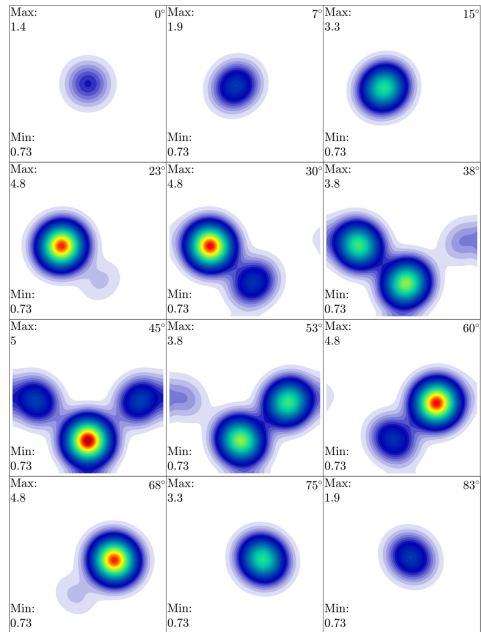
```
rec2 = calcODF(pf{1:2})
```

plot C coefficients

```
plotFourier(SantaFe)
```

```
plotFourier(rec)
```

```
plotFourier(rec2)
```



The Even / Odd Problem

the famous SantaFe sample odf

```
plot(SantaFe, 'layout', [4 3])
```

simulate discrete pole figures

```
pf = calcPoleFigure(odf, h)
```

odf reconstruction without ghost correction

```
rec = calcODF(pf, 'noGhostCorrection')
```

odf reconstruction with ghost correction

```
rec6 = calcODF(pf)
```

odf reconstruction with 2 pole figures

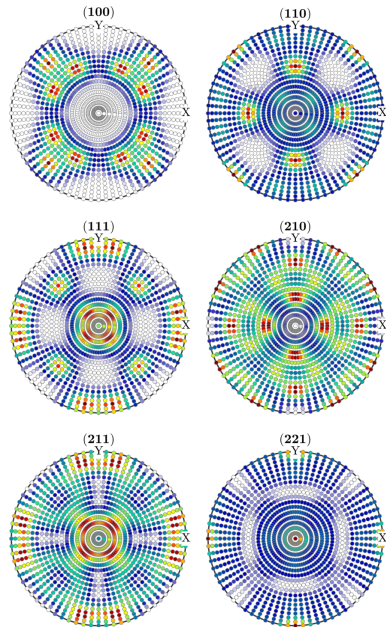
```
rec2 = calcODF(pf{1:2})
```

plot C coefficients

```
plotFourier(SantaFe)
```

```
plotFourier(rec)
```

```
plotFourier(rec2)
```



The Even / Odd Problem

the famous SantaFe sample odf

```
plot(SantaFe , 'layout' , [4 3])
```

simulate discrete pole figures

```
pf = calcPoleFigure(odf , h)
```

odf reconstruction without ghost correction

```
rec = calcODF(pf , 'noGhostCorrection')
```

odf reconstruction with ghost correction

```
rec6 = calcODF(pf)
```

odf reconstruction with 2 pole figures

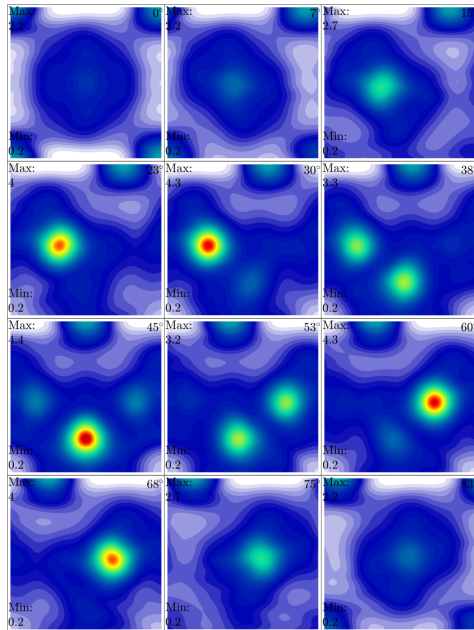
```
rec2 = calcODF(pf{1:2})
```

plot C coefficients

```
plotFourier(SantaFe)
```

```
plotFourier(rec)
```

```
plotFourier(rec2)
```



The Even / Odd Problem

the famous SantaFe sample odf

```
plot(SantaFe, 'layout', [4 3])
```

simulate discrete pole figures

```
pf = calcPoleFigure(odf, h)
```

odf reconstruction without ghost correction

```
rec = calcODF(pf, 'noGhostCorrection')
```

odf reconstruction with ghost correction

```
rec6 = calcODF(pf)
```

odf reconstruction with 2 pole figures

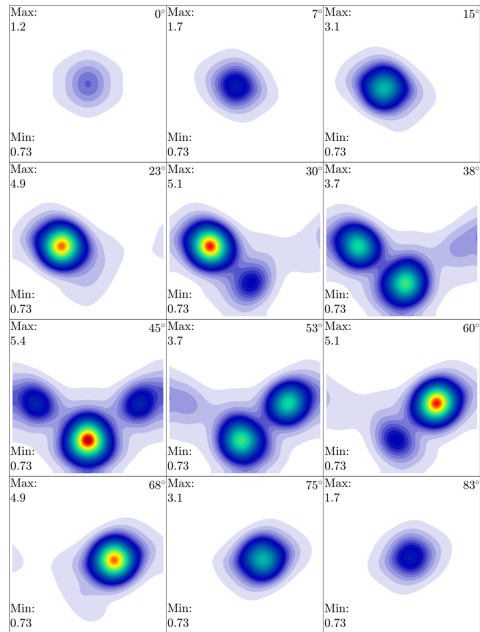
```
rec2 = calcODF(pf{1:2})
```

plot C coefficients

```
plotFourier(SantaFe)
```

```
plotFourier(rec)
```

```
plotFourier(rec2)
```



The Even / Odd Problem

the famous SantaFe sample odf

```
plot(SantaFe , 'layout' , [4 3])
```

simulate discrete pole figures

```
pf = calcPoleFigure(odf , h)
```

odf reconstruction without ghost correction

```
rec = calcODF(pf , 'noGhostCorrection')
```

odf reconstruction with ghost correction

```
rec6 = calcODF(pf)
```

odf reconstruction with 2 pole figures

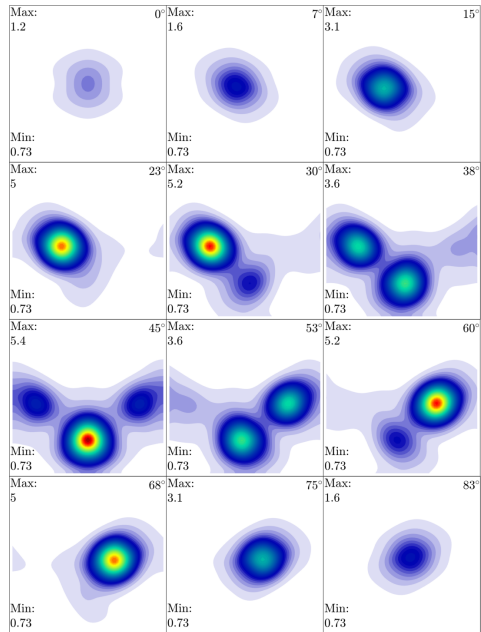
```
rec2 = calcODF(pf {1:2})
```

plot C coefficients

```
plotFourier(SantaFe)
```

```
plotFourier(rec)
```

```
plotFourier(rec2)
```



The Even / Odd Problem

the famous SantaFe sample odf

```
plot(SantaFe, 'layout', [4 3])
```

simulate discrete pole figures

```
pf = calcPoleFigure(odf, h)
```

odf reconstruction without ghost correction

```
rec = calcODF(pf, 'noGhostCorrection')
```

odf reconstruction with ghost correction

```
rec6 = calcODF(pf)
```

odf reconstruction with 2 pole figures

```
rec2 = calcODF(pf{1:2})
```

plot C coefficients

```
plotFourier(SantaFe)
```

```
plotFourier(rec)
```

```
plotFourier(rec2)
```

